

PerfMLOps Case Study Test Results

I. RESULTS

This section presents the outcomes of the experiments conducted in this study. These results were further analyzed to evaluate the effectiveness of our approach to identifying performance bottlenecks and their root causes using BigQuery's in-built Machine learning model. The results also provide insights into model performance and system behavior, laying the foundation for interpreting findings and discussing their implications in the context of the research objectives.

A. API Operational Insights

This section provides results obtained after the successful deployment of the Python API on a local computer. After the successful deployment, the API is available over the internet on the local network, triggered the JMeter test, and sent the results via email,

B. JMeter Performance Test Results

Requests were generated using Apache JMeter to simulate traffic to the deployed API based on predefined test data containing the fields Name, Description, Price, Quantity, Update item, and Endpoint. The requests were distributed across multiple endpoints, including fast, slow, random (with response times varying between 0.1 and 3 seconds), and unstable (returning intermittent 500 errors). This section presents the results of the load test execution conducted with a sustained load of 50 transactions per second (TPS) for one hour, including response time visualizations and the results of the intentionally induced performance bottlenecks.

The email received from JMeter via Python code displays test results along with the counts of passed and failed transactions.

C. Events in Google Analytics

The report from GA4, sent in the email, includes the number of events that were sent to it. The number of events that were sent matched the number of GA requests sent during the tests.

The results indicate that the integration between the client-side test setup and GA was consistent throughout multiple runs. Furthermore, this confirms that the event dispatching logic was working as expected without introducing duplication or missing events.

D. Data from BigQuery Tables

Tables in BigQuery show the number of rows and the performance bottlenecks that were inserted into the BigQuery table, allowing us to use them to train the models. Requests

with non-200 response codes or response times over 450ms are counted as bottlenecks. The insertion counts align with the volume of events captured from Google Analytics.

Inference Query Results from the ML Model summarize how many performance bottlenecks were detected in a JMeter test run and how confident the ML model was in those detections. avg_prob is the Average probability assigned by the ML model that these events are bottlenecks (model confidence level).

E. Test Summary from Cloud Function

The email received from the Cloud Function with Test results shows the email sent by the Python code written in Google's cloud function, including a test summary, the number of bottlenecks identified, and their root causes.

II. ANALYSIS

Google Analytics and BigQuery successfully sent all the events generated from JMeter and posted the status via email in each stage of the pipeline automatically, without any noticeable delay. The BigQuery ML model accurately identified the intentionally induced bottlenecks, with predicted metrics closely aligning with the observed JMeter results. The cloud function took some time to deploy the service for the first time and sent emails with BigQuery table results without any noticeable delay. The end-to-end framework operated in a fully automated manner with no human intervention, except for the initial system setup and occasional investigations when anomalies were detected.

The data in the table I show that, out of 6,050 requests sent by JMeter, 6,032 were successfully processed downstream, indicating minimal ingestion loss. While JMeter reports a high pass rate of 99.40% with only 36 failures at the request level, BigQuery identifies more failures (305) due to additional validation conditions, reducing the pass rate to 94.67%. The ML model further increases failure detection to 503 cases by identifying potential bottlenecks and patterns beyond rule-based checks, resulting in a 90.90% pass rate, which is consistently reflected in Cloud Runs email-triggering stage. Overall, the increase in detected failures across stages reflects progressively stricter analysis rather than system issues, showing that downstream components apply deeper validation and predictive assessment.

The estimated reductions in deployment cycle time 75% and performance-related incidents 90% are derived from simulations conducted in this controlled environment, benchmarked against typical manual/semi-automated performance testing workflows reported in industry studies [?] [?]. In our

experiments, the fully autonomous pipeline achieved end-to-end feedback in 15 minutes (from deployment trigger to anomaly/root-cause email), compared to multi-hour to multi-day cycles common in manual JMeter scripting, execution, log analysis, and ad-hoc bottleneck investigation.

TABLE I
TABLE SHOWING EVENTS CAPTURED IN EACH LAYER

	Requests Count	Events Sent	Passed	Failed	Pass %
Sent by JMeter	6050	848	6014	36	99.4%
Received by GA4	NA	845	NA	NA	NA
Received by BigQuery	6032	NA	5727	305	94.67%
Identified by ML Model	6032	NA	5529	503	90.9%
Identified by CloudRun to Send Email	6032	NA	5529	503	90.9%